4.c Classes - Extensions

Extending Kotlin Classes Without Inheritance

Contents

1 **Extension Functions**
Syntax, receiver types, this keyword, interfaces & generics

2 **Extension Properties**
Computed properties, no backing fields, nullable receivers

3 **Resolution & Dispatch**
Static resolution, member wins rule, companion object extensions

4 **Scope & Visibility**
Member extensions, dispatch vs extension receiver, visibility rules

What Are Extensions?

Extensions let you add new functionality to a class or interface without inheritance or the Decorator pattern. You call them as if they were original members.

Key concepts about extensions:

Extension Functions

Add new callable functions to any class using receiver type + dot notation. The **this** keyword refers to the receiver object.

Extension Properties

Add computed properties to existing classes. No backing fields allowed - must use explicit getters and setters.

Resolved Statically

Extensions are dispatched based on the declared type at compile time, not the runtime type. They are not virtual.

Don't Modify Classes

Extensions don't insert new members into a class. They are syntactic sugar making functions callable with dot notation.

Extension Functions – declaration and usage

To declare an extension function, prefix its name with the receiver type (the class being extended) followed by a dot.

Syntax:

```
fun String.truncate(maxLength: Int): String {
    return if (this.length <= maxLength) this else take(maxLength - 3) + "..."
}
```

Usage:

```
fun testExtensions1() {
    val text = "This is a long string that needs to be truncated."
    println(text.truncate(20))   // Output: "This is a long st..."
}
```

Key Points:

- The `this` keyword inside an extension function refers to the receiver object
- Works on both user-defined classes and standard library classes (String, Int, List, etc.)
- Extensions are defined outside the class, typically at the top level under packages

Extensions on Interfaces & Generics

Extending Interfaces

Add functionality to all types implementing an interface:

```
interface User {
    val name: String; val email: String
}

fun User.displayInfo(): String = "User($name, $email)"

val user = object : User {  // instance of an anonymous class
    override val name: String = "Alice"
    override val email: String = "alice@bmail.com"
}

println(user.displayInfo())
// output: User(Alice, alice@bmail.com)
```

Generic Extensions

Declare a generic-type parameter before the function name:

```
fun <T> MutableList<T>.swap(
    idx1: Int, idx2: Int
) {
    val tmp = this[idx1]
    this[idx1] = this[idx2]
    this[idx2] = tmp
}

val list = mutableListOf(1, 2, 3)
println(list)   // Output: [1, 2, 3]
list.swap(0, 2)
println(list)   // Output: [3, 2, 1]
```

Extending an interface adds the function to **all types** that implement it. Generic type parameters make extensions reusable across types.

Standard Library Extensions

- Many stdlib functions (removePrefix(), uppercase(), filter()) are extension functions
- Extensions enable call chaining: **title.removePrefix("The ").uppercase()**

Extension Properties

Extension properties add computed properties to existing classes. Since extensions don't insert members, there is no backing field - you must provide explicit getters/setters.

Read-only (val):

```
val <T> List<T>.lastIndex: Int
    get() = size - 1

// Usage:
listOf(1, 2, 3).lastIndex // 2
```

Mutable (var):

```
class Temperature(var celsius: Float)

var Temperature.fahrenheit: Float
    get() = (celsius * 9 / 5) + 32
    set(value) {
        celsius = (value - 32) * 5 / 9
    }

val temp = Temperature(25f)
println("Celsius: ${temp.celsius}")
println("Fahrenheit: ${temp.fahrenheit}")

temp.fahrenheit = 86f
println("Celsius: ${temp.celsius}")
println("Fahrenheit: ${temp.fahrenheit}")
```

```
Output:
Celsius: 25.0
Fahrenheit: 77.0
Celsius: 30.0
Fahrenheit: 86.0
```

Important Rules:

- Initializers are **not allowed** for extension properties (no backing fields)
- Behaviour must be defined by explicit getters (and setters for var)
- Best suited for computed/derived values - avoid heavy computation in getters

```
// Won't compile:
var House.number = 1  // Error: Initializers not allowed
```

Nullable Receiver

Extensions can be defined with a nullable receiver type, allowing them to be called on variables even if their value is null. When the receiver is null, **this** is also null.

Example:

```
fun Any?.safeToString(): String {
    if (this == null) return "null"
    // After null check, 'this' is smart-cast to Any
    return this.toString()
}

val a: Any? = null
val b: Any? = 42
val c: Any? = "Kotlin"

println(a.safeToString())
println(b.safeToString())
println(c.safeToString())
```

```
Output:
null
42
Kotlin
```

Key Takeaways:

- Receiver type uses **?** suffix (e.g. **Any?**, **String?**)
- After a null check, **this** is smart-cast to a non-null type automatically
- Kotlin's **toString()** uses this pattern - you can call it without checking for null

Extensions Are Resolved Statically

Extension functions are resolved based on the **declared type** of the variable, not the runtime type. They do not support polymorphism.

Example:

```
open class Shape
class Rectangle : Shape()

fun Shape.getName() = "Shape"
fun Rectangle.getName() = "Rectangle"

fun printName(s: Shape) = println(s.getName())
printName(Rectangle())   // Prints "Shape", NOT "Rectangle"
```

Key Takeaways:

- The parameter type (**s: Shape**) determines which extension is called at compile time
- Extensions do **not** participate in virtual dispatch - they are compiled as static functions
- If you need polymorphic behaviour, use member functions or interfaces instead

Member Always Wins

If a class has a member function with the same signature as an extension function, the **member always wins**.

Example:

```
class Example {
    fun printMessage() { println("Class member") }
}

fun Example.printMessage() { println("Extension function") }

Example().printMessage()  // Prints "Class member"
```

Key Takeaways:

- A member function with the same signature always takes precedence over an extension
- Extensions can **overload** member functions (same name, different parameters)
- The compiler produces a warning if an extension shadows a member function

Companion Object Extensions

If a class has a companion object, you can define extension functions and properties on it. Call them using the class name as the qualifier.

Example:

```
class MyClass {
    companion object { }
}

fun MyClass.Companion.printCompanion() { println("companion") }

MyClass.printCompanion()  // Called on the class itself
```

Key Takeaways:

- The class **must** define a companion object (even if empty) for this to work
- Called using the class name as qualifier: **MyClass.printCompanion()**
- Useful for adding factory methods or utility functions to existing classes

Member Extensions

Extensions can be declared as members of another class. Inside such an extension, there are two implicit receivers: the **dispatch receiver** (enclosing class) and the **extension receiver**.

Example:

```
class Connection(val host: String, val port: Int)

class Host {

    fun Connection.getConnectionString(): String {
        return "$host/$port"
        // host and port from Connection (extension receiver)
        // Host is the dispatch receiver
    }

    fun connect(connection: Connection) {
        println(connection.getConnectionString())
    }
}
```

```
val host = Host()
val connection = Connection("localhost", 8080)
host.connect(connection)
```

```
Output:
localhost/8080
```

Key Takeaways:

- Use **this@Host** to reference the dispatch receiver, **this@getConnectionString** for the extension receiver
- Member extensions can be declared **open** and overridden in subclasses (dispatch receiver is virtual)

Scope & Visibility

Extensions are usually defined at the top level, directly under packages. To use them outside the declaring package, import them at the call site.

Importing Extensions:

```
import com.example.utils.truncate
import com.example.utils.*        // or wildcard import
```

Visibility Rules:

- Extensions can use all **public** and **protected** members of the receiver class
- Extensions cannot access **private** or **internal** members (they don't break encapsulation)
- The extension itself can have visibility modifiers: **private**, **internal**, **public** (default)
- A **private** top-level extension is visible only in the file where it is declared
- Member extensions in a class are scoped to that class and cannot be called from outside

Practice Exercises

Solved Exercise 1

Task: Write an extension function **List<Int>.secondOrNull()** that returns the second element, or **null** if the list has fewer than 2 elements.

Solution:

```
fun List<Int>.secondOrNull(): Int? {
    return if (this.size >= 2) this[1] else null
}
```

Usage:

```
listOf(10, 20, 30).secondOrNull()   // 20
listOf(1).secondOrNull()            // null
emptyList<Int>().secondOrNull()     // null
```

Concepts Used:

Extension function • Receiver type • **this** keyword • Nullable return type

Solved Exercise 2

Task: Write an extension property **String.isPalindrome** that returns **true** if the string reads the same forwards and backwards (case-insensitive).

Solution:

```
val String.isPalindrome: Boolean
    get() {
        val cleaned = this.lowercase()
        return cleaned == cleaned.reversed()
    }
```

Usage:

```
"racecar".isPalindrome     // true
"Kotlin".isPalindrome      // false
"Level".isPalindrome       // true (case-insensitive)
```

Concepts Used:

Extension property • Custom getter • No backing field • **this** keyword

Unsolved Exercises

1 Int.isEven

Write an extension property Int.isEven that returns true if the integer is even.

Hint: Use the modulo operator (%) on this

2 String?.orDefault()

Write an extension function on String? that returns the string or a default value if null.

Hint: Use a nullable receiver type

3 MutableList<T>.swap()

Write a generic extension function that swaps two elements at given indices in a MutableList.

Hint: Use a generic type parameter <T>

4 Companion Factory

Add an extension function on a User.Companion that creates a User from a formatted string.

Hint: The class needs a companion object { }

Challenge Exercise:

5 DSL-Style Builder with Member Extensions

Create an **HtmlBuilder** class that uses member extensions on **String** to wrap text in HTML tags (e.g. **"Hello".bold()**, **"text".italic()**).

Hint: Use member extensions so bold() and italic() only work inside the builder scope